

Aluno: Maicon Heuberth Alves Coutinho.

Documentação do jogo desenvolvido no Trabalho TP_ALLEGRO da matéria de Programação e Desenvolvimento de Software I

Módulo: Jogo de computador "Chain Blast"

VISÃO GERAL:

Este código foi escrito com a finalidade de criar uma versão simplificada do jogo "Chain Blast", no qual inimigos vivos são destruídos quando colidem com inimigos destruídos pelo herói.

PRÉ-REQUISITOS E INSTALAÇÃO:

Para poder rodar esse programa são necessários:

- Possuir o compilador gcc pré-instalado na sua máquina;
- Possuir a biblioteca allegro5 instalada em seu PC;
- Possuir os arquivos Makefile, background.jpg e title_music.ogg que acompanham o arquivo chain.c.

GUIA DE USO:

Para rodar o código siga as seguintes instruções:

1. Abra o terminal e vá para a pasta onde está o arquivo TP_ALLEGRO.zip;
2. Extraia o arquivo TP_ALLEGRO.zip;
3. No terminal, digite o comando "make";
4. Digite "./chain";
5. Considerações: O arquivo Makefile foi criado pensando no Linux, porém possui os comandos para Windows comentados com "#". Caso sua máquina seja Windows, edite o Makefile retirando os comentários da segunda parte (para Windows) e comentando os comandos do Linux.

DESCRIÇÃO DAS FUNÇÕES/MÓDULOS:

O código foi escrito em Módulos para facilitar a visualização e a organização. Seguem a seguir os módulos e explicações sobre bibliotecas incluídas.

BIBLIOTECAS:

Foram usadas as seguintes bibliotecas padrão da linguagem C:

- <stdio.h> e <stdlib.h>: Bibliotecas utilizadas para entrada e saída de dados e funções utilitárias (como rand() para geração de números aleatórios).
- <math.h> : Biblioteca usada para realizar operações matemáticas.
- <time.h> : Usada para inicializar a semente da geração de números aleatórios. Como (srand(time(NULL))) que garante que os inimigos surjam em posições aleatórias.

Módulos da allegro5:

- <allegro5/allegro.h>: Responsável pelo loop principal, gerenciamento de janelas(display), tempo(timers) e eventos(funcionamento dos botões do teclado);
- <allegro5/allegro_primitives.h>: Permite desenhar formas geométricas na tela, como o triângulo do jogador;
- <allegro5/allegro_font.h> e allegro5/allegro_ttf.h : Permitem renderizar textos na tela como o placar e o recorde;
- <allegro5/allegro_image.h>: Permite o uso de imagens externas como background.jpg;
- <allegro5/allegro_audio.h> e <allegro5/allegro_acodec.h>: Permite carregar e acessar áudio externo (title_music.ogg);

DEFINIÇÃO DE MACROS:

- NUM_ENEMIES : Número máximo de inimigos na tela;
- TIRO_INATIVO, TIRO_ATIVO e TIRO_RECARREGANDO : Usados para desativar, ativar e colocar o tiro no modo recarregando, respectivamente;
- RAIO_TIRO : Define o raio de distância do tiro;
- BORDA_TIRO : Define a espessura do campo de força;
- SCORE_PENALTY : Define a penalidade que o jogador sofre com o tempo;

CONSTANTES GLOBAIS:

- FPS : Define a quantidade de imagens corridas por segundo, no caso, 100;
- SCREEN_W : Tamanho máximo da tela na horizontal;
- SCREEN_H : Tamanho máximo da tela na vertical;
- HERO_W e HERO_H : Largura e altura do herói, respectivamente;

VARIÁVEIS GLOBAIS:

- ALLEGRO_BITMAP : Cria um ponteiro para a criação da imagem de fundo;
- ALLEGRO_COLOR BKG_COLOR : Responsável pelas cores do jogo;
- ALLEGRO_FONT *FONT_32: Responsável pela fonte dos textos impressos no jogo;

DEFINIÇÃO DE ESTRUTURAS:

Tiro: Possui os campos x e y, raio, modo, timer e cor, responsáveis pela localização do tiro, pelo raio do círculo, pelo modo(ativo, inativo ou recarregando), pelo tempo de tiro e pela cor do tiro, respectivamente;

Ship: Possui os campos x, y, vel, cor e tiro, responsáveis pela localização dos personagens, pela velocidade, cor e se atiram ou não;

Hero: Campos: Ship, dir_x, dir_y e score, responsáveis pela posição, direção do movimento e pontuação;

Enemy: Campos: Ship, raio, active, explosao, raio_da_explosao e tempo_da_explosao, responsáveis pela localização, raio dos inimigos, se está ativo ou inativo, pela explosão, pelo raio dessa explosão e tempo de duração da explosão.

FUNÇÕES CRIADAS:

- initTiro: recebe a posição do tiro como parâmetro e o inicializa;
- initGlobals: Inicializa a fonte e as cores e não recebe parâmetros;
- carregaRecorde: Não recebe parâmetros e cria/abre o arquivo recorde.txt e checa se o arquivo é nulo. Retorna o recorde atual;
- salvaRecorde: Recebe como parâmetro a variável novo_recorde, abre o arquivo recorde.txt e escreve o novo recorde lá, checando também se o arquivo não é nulo. Não retorna valor;
- initHero : Recebe como parâmetro uma estrutura Hero e inicializa cada um de seus membros, não retorna valor;
- initEnemy : Recebe como parâmetro um ponteiro para uma estrutura inimigo, inicializa cada campo dela e não retorna valor;

- drawScenario : Recebe uma estrutura Heroi como parâmetro, desenha a imagem de fundo e cria um vetor de pontuação (score_txt). Não retorna valor;
- drawHero: Recebe um ponteiro para uma estrutura Hero como parâmetro, desenha o triângulo do herói na tela, desenha o tiro e escreve a pontuação. Não retorna valor;
- updateEnemies : Recebe um vetor de inimigos e atualiza inimigo por inimigo, checando se está ativo ou não, se está explodindo ou não e calcula o crescimento da explosão. Essa função não retorna valor;
- updateHero : Função que recebe uma estrutura Hero, checa se o herói não ultrapassa os limites da tela, atualiza a posição dele e atualiza o modo de tiro;
- drawEnemy : Recebe um vetor enemies[] de estruturas Enemy, checa se o vilão está explodindo ou não e desenha ele na tela. Não retorna valor;
- checaColisao : Recebe o centro (x, y) e o raio dos inimigos e do herói, calcula a distância entre os centros e retorna 1 se houver colisão e 0 se não houver;

FUNÇÃO MAIN: FLUXO PRINCIPAL DO JOGO:

A função main foi dividida em três tópicos com o objetivo de facilitar a visualização e a posterior alteração do código quando necessário. A seguir, segue a explicação detalhada dos tópicos:

INICIALIZAÇÃO:

recorde_quebrado : Variável que guarda a informação da quebra ou não do recorde;

*display : variável inicializada como NULL que guarda as informações gráficas da tela;

*event_queue : É um ponteiro para a fila de eventos servido para organizar cronologicamente os eventos do jogo. É inicializada como NULL;

*timer : Ponteiro que armazena as informações relativas ao tempo de jogo. Também inicializada como NULL;

*musica_de_fundo : Ponteiro para o arquivo que armazena a música de fundo. Também inicializado como NULL;

al_install_audio() : Função que instala todos os áudios no sistema;

al_init_acodec_addon() : Liga o decodificador de som;

al_reserve_samples(4) : Reserva espaço para no máximo 4 sons no sistema;

al_load_sample("title_music.ogg") : Armazena a música e inicializa ela;

playing : Variável usada para checar se o jogo está ativo ou não;

frames_contados : Variável usada para contar os frames. Muito útil na contagem de tempo da morte de um inimigo;

proximo_vilao : Variável usada para colocar na tela um novo inimigo;

Enemy enemies[NUM_ENEMIES] : Declara um vetor de inimigos e em seguida inicializa cada um num loop for;

al_play_sample() : Começa a tocar a música de fundo;

al_load_bitmap() : Carrega a imagem de fundo;

LOOP PRINCIPAL : WHILE(PLAYING)

Esse loop armazena um evento e após usa ele nas checagens:

ALLEGRO_KEY_DOWN : Macro própria da allegro5 que significa "tecla pressionada";

switch(ev.keyboard.keycode) : Checa qual foi a tecla pressionada e dá o movimento ao personagem(keys W, S, A, e D), o tiro(key SPACE) e o tiro duplo (key Z);

ALLEGRO_KEY_UP : Macro que significa "tecla não pressionada". Ela faz com que o personagem cesse o movimento;

if(ev.type == ALLEGRO_EVENT_TIMER) : Trecho do loop responsável pelas funções relacionadas ao tempo:

Atualizar personagens (updateHero e updateEnemies),

Macro SCORE_PENALTY, responsável por aplicar a penalidade de tempo no herói;

Ativar inimigos com blocos if;

Checar se algum tiro acertou um vilão, se o vilão colidiu com outros e se o herói colidiu com um vilão vivo e aumentar os pontos ou terminar o jogo;

redesenhar : Variável usada para redesenhar a cena a cada frame caso houver algum evento. Isso é checado num bloco if logo após;

Processamento de fim de jogo:

al_rest(3) : Dá uma pausa de 3 segundos na tela caso o herói morrer;

my_text[100] : Vetor que guarda a pontuação do jogo;

al_draw_text() : Variável que, associada ao sprintf desenha a pontuação na tela com uma formatação, cor e tamanho especificados;

if (recorde_quebrado == 1) : Bloco que checa se o recorde foi quebrado e escreve "PARABÉNS" na tela caso tenha sido quebrado;

text_recorde[100] : Vetor que guarda o recorde atual que é mostrado na tela logo após;

al_flip_display() : Função que atualiza a tela;

al_destroy_timer() e análogos: Fecham cada arquivo/objeto usados no jogo;

return 0 : Retorna 0 fechando a função main com sucesso.